

Λειτουργικά Συστήματα

Project 3

Τσόγκας Παναγιώτης	Νίκας Ιωάννης Ιάσων	Τσιαντός Δημήτριος
ptsogkas@uth.gr	ioannikas@uth.gr	dtsiantos@uth.gr
3672	3771	3796

Ιούνιος 2025

Περιεχόμενα

1	Περιγραφή	3
2	Βασικές Λειτουργίες Αρχείων	3
2.1	Εγγραφή	3
2.2	Ανάγνωση	3
2.3	Διαγραφή	4
3	Ιεραρχία Φακέλων και Αρχείων	5
4	Συναρτήσεις	6
5	Πειράματα	6
5.1	Έλεγχοι Δομών Blocks	6
5.2	Έλεγχοι Δομών Ιεραρχίας Καταλόγων και Αρχείων	7
6	Οδηγίες Εκτέλεσης	7
6.1	Εκτέλεση Συστήματος Αρχείων	7
6.2	Εκτέλεση Πειραμάτων	7

1 Περιγραφή

Στην παρούσα εργασία παρουσιάζουμε την υλοποίηση ενός non-volatile συστήματος αρχείων με συμπίεση (compression) σε επίπεδο block. Αυτό μπορεί να μειώσει τον δεσμευμένο χώρο στον δίσκο σε περίπτωση που έχουμε αρχεία με ίδια blocks αυξάνοντας, ωστόσο, το κόστος των εγγραφών, των αναγνώσεων και των διαγραφών σε σύγκριση με ένα απλό σύστημα αρχείων.

Στόχος του συστήματος αρχείων είναι η επαναχρησιμοποίηση των κοινών blocks, η επιτυχής επανακατασκευή του αρχικού αρχείου κατά την **ανάγνωση**, η σωστή ανανέωση των μεταδεδομένων κατά την **εγγραφή** και την **διαγραφή** καθώς και η διατήρηση της δομής των **αρχείων** και των **φακέλων**.

Για λόγους απλότητας και ευκολίας η υλοποίηση του file system βασίζεται πάνω στο Big Brother File System (BBFS) που χρησιμοποιεί την βιβλιοθήκη fuse, αξιοποιώντας το Virtual File System.

2 Βασικές Λειτουργίες Αρχείων

2.1 Εγγραφή

Για να καταφέρουμε να συμπίεσουμε τα αρχεία κατά την εγγραφή, αρκεί i) να εντοπίσουμε τα ίδια blocks που γράφονται στον δίσκο, ii) να τα γράψουμε μία μόνο φορά και iii) να κρατήσουμε δείκτες για το κάθε αρχείο που να δείχνουν σε αυτά τα blocks.

Για το πρώτο βήμα χρησιμοποιούμε τον αλγόριθμο hashing SHA-1 ώστε να αναγνωρίσουμε τα κοινά blocks δεδομένων. Να σημειωθεί πως μπορεί να υπάρξει το φαινόμενο του hash collision, κάτι που αγνοούμε σε αυτή την υλοποίηση. Εφόσον εντοπίσουμε ένα νέο block, το γράφουμε στην 'αποθήκη' (repository) των blocks σε συνεχόμενες θέσεις και τοποθετούμε μία νέα εγγραφή σε ένα hashmap με κλειδί το hash και με τιμή το offset του block. Αυτό το hashmap θα διατηρεί μεταδεδομένα για τα blocks.

Hash (SHA-1)	Block offset
01b307acba4f54f55aafc33bb06bbbf6ca803e9a	3
bd5e5eb049f3907175f54f5a571ba6b9fdea36ab	1
64ea0dc7dadd49a337f1ef14815bd3f428141c7d	8
...	...

Πίνακας 1: Παράδειγμα hashmap με αντιστοιχίες block hashes σε blocks offset του block repository.

Επιλέγουμε τα 4KB ως το σταθερό μέγεθος του κάθε block και 4 bytes για το μέγεθος του πεδίου block offset. Αυτό σημαίνει πως σε αυτό το σύστημα αρχείων μπορούμε να γράψουμε μέχρι και 2^{32} μοναδικά blocks. Δηλαδή η μέγιστη χωρητικότητα διαφορετικών blocks στον δίσκο θα είναι:

$$\#max_blocks = 2^{block_offset_bits} = 2^{32} = 4,294,967,296\ blocks$$

$$max_disk_size = \#max_blocks \times sizeof_block = 2^{32} \times 2^{12} = 2^{44} = 16TB$$

2.2 Ανάγνωση

Προκειμένου να διαβάσουμε ένα αρχείο χρειάζεται να ανακατασκευάσουμε τα επιμέρους blocks του. Αυτό μπορεί να επιτευχθεί αποθηκεύοντας την σειρά των blocks σε ένα εικονικό αρχείο κατά την εγγραφή.

Για την ανάγνωση δυναμικού μεγέθους, και όχι μόνο πολλαπλάσια μεγέθη των τεσσάρων kilobytes, είναι απαραίτητο να αποθηκεύουμε το μέγεθος του τελευταίου block, καθώς μόνο το τελευταίο είναι εκείνο που μπορεί να έχει μέγεθος μικρότερο από 4KB. Το δυναμικό μέγεθος αρχείων μπορεί να προκαλέσει εσωτερικό κατακερματισμό στην περίπτωση που το μέγεθος των bytes δεν είναι πολλαπλάσιο του 4096 αλλά στην χειρότερη περίπτωση ο κατακερματισμός αυτός θα είναι 4095 bytes για κάθε αρχείο του συστήματος και κατά μέσο όρο περίπου 2048 bytes ανά αρχείο.

Last Block Size	50
Hashses	01b307acba4f54f55aafc33bb06bbbf6ca803e9a 01b307acba4f54f55aafc33bb06bbbf6ca803e9a bd5e5eb049f3907175f54f5a571ba6b9fdea36ab 64ea0dc7dadd49a337f1ef14815bd3f428141c7d 64ea0dc7dadd49a337f1ef14815bd3f428141c7d ...

Πίνακας 2: Παράδειγμα εικονικού αρχείου που χρησιμοποιείται για την ανακατασκευή του αρχικού με χρήση του μεγέθους του τελευταίου block και των block hashes.

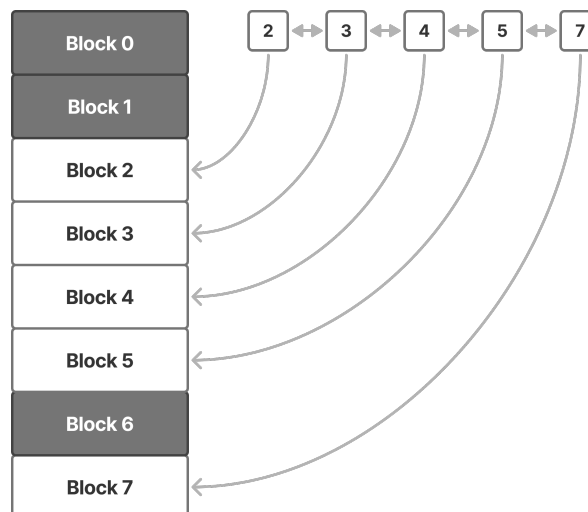
2.3 Διαγραφή

Κατά την διαγραφή μπορεί να προκύψουν κάποια προβλήματα στο hashmap και στο repository. Αφού γίνει η διαγραφή ενός αρχείου, δηλαδή η κατάργηση του εικονικού αρχείου, είναι απαραίτητο να διαγράψουμε και τα αντίστοιχα blocks που χρησιμοποιούσε. Μπορεί, ωστόσο, άλλα εικονικά αρχεία να χρειάζονται αυτά τα blocks. Η λύση σε αυτό το πρόβλημα είναι να τοποθετήσουμε στο blocks hashmap μία ακόμα τιμή που να γράφει τον αριθμό των αναφορών από άλλα αρχεία προς κάθε block. Με αυτή την αλλαγή θα διαγράψουμε ένα block, μόνο σε περίπτωση που οι αναφορές προς αυτό είναι μηδέν.

Hash (SHA-1)	Block offset	References
01b307acba4f54f55aafc33bb06bbbf6ca803e9a	3	2
bd5e5eb049f3907175f54f5a571ba6b9fdea36ab	1	1
64ea0dc7dadd49a337f1ef14815bd3f428141c7d	8	2
...	...	...

Πίνακας 3: Παράδειγμα hashmap με αντιστοιχίες block hashes σε blocks offset του block repository.

Ύστερα από διαγραφές αρχείων, και κατ' επέκταση blocks, δημιουργούνται κενά μέσα στο repository που ιδανικά θα θέλαμε να ξανά γεμίσουν τοποθετώντας νέα blocks από επόμενες εγγραφές.



Σχήμα 1: Αποθήκη blocks και λίστα ελεύθερων θέσεων.

Για να γνωρίζει το σύστημα ποια blocks είναι ελεύθερα, θα χρειαστεί να αποθηκεύουμε μία ταξινομημένη διπλά διασυνδεδεμένη κυκλική λίστα με sentinel που θα έχει για τιμές τα offsets των ελεύθερων

blocks. Έτσι, κατά την εγγραφή, βγάζουμε το πρώτο στοιχείο από την λίστα και χρησιμοποιούμε το offset ώστε να γράψουμε στο αντίστοιχο block.

Άλλο ένα πρόβλημα που μπορεί να προκύψει από την διαγραφή είναι ο εξωτερικός κατακερματισμός στην αποθήκη των blocks. Στην χειρότερη περίπτωση, σε ένα σύστημα γεμάτο από μοναδικά blocks όταν διαγραφούν όλα εκτός από το τελευταίο, αν έρθει μία νέα εγγραφή δεν θα υπάρχει διαθέσιμος χώρος για τα μεταδεδομένα καθώς η αποθήκη των blocks θα εξακολουθεί να έχει το αρχικό σταθερό μέγεθος. Αυτό λύνεται με ανασυγκρότηση (defragmentation) των blocks, γράφοντας τα τελευταία στις πρώτες ελεύθερες θέσεις.

Ιδανικά η συνάρτηση που εκτελεί αυτή την διαδικασία θα έπρεπε να καλείται υπό ορισμένες συνθήκες όπως, για παράδειγμα, αν ο δίσκος αρχίζει να γεμίζει. Για δοκιμαστικούς σκοπούς, σε αυτή την υλοποίηση εκτελούμε μερική ανασυγκρότηση όταν ο εξωτερικός κατακερματισμός ξεπεράσει το 10%, μέχρι να φτάσει κάτω από 10% κρατώντας κάθε χρονική στιγμή το ποσοστό κάτω από αυτό το όριο. Το πρόβλημα με αυτή την τακτική είναι ότι εμφανίζει το κλασικό πρόβλημα του «θερμοστάτη», σύμφωνα με το οποίο, όταν οι τιμές του κατακερματισμού βρίσκονται στο όριο της επιλεγμένης τιμής, το σύστημα τείνει να «ταλαντώνεται» γύρω από αυτό το όριο.

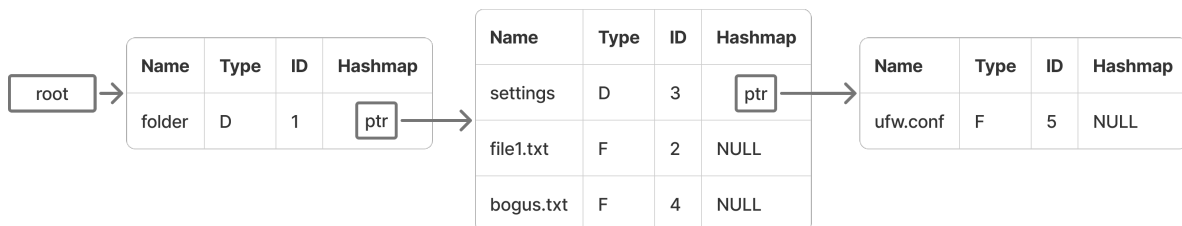
3 Ιεραρχία Φακέλων και Αρχείων

Άλλη μία βασική ευθύνη του συστήματος αρχείων είναι να δείχνει στον χρήστη μια δομημένη δενδροειδή ιεραρχία φακέλων και αρχείων. Η προσωμοίωση μίας τέτοιας δομής απαιτεί να αποθηκεύσουμε μεταδεδομένα για τους φακέλους. Ο κάθε φάκελος θα έχει το δικό του hashmap με τα περιεχόμενά του, δηλαδή τα ονόματα των κόμβων που περιέχει, τον τύπο του κόμβου (αρχείο ή φάκελος), το ξεχωριστό κλειδί (id) του κάθε κόμβου και, στην περίπτωση φακέλου, έναν δείκτη σε άλλο hashtable.

ID	Type	Name Length	Name
2	F	9	file1.txt
3	D	9	settings
4	F	5	bogus
...	...	...	...

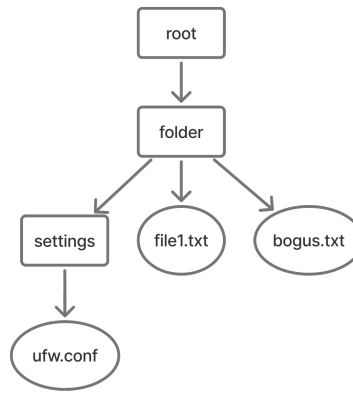
Πίνακας 4: Παράδειγμα μεταδεδομένων ενός φακέλου στον δίσκο.

Μπορούμε να αποθηκεύσουμε τα δεδομένα αυτά στον δίσκο και όταν το filesystem αρχικοποιείται να φτιάχνουμε αναδρομικά την ιεραρχία των φακέλων στην μνήμη για καλύτερη επίδοση. Όταν το file system αποσυνδεθεί από το υπόλοιπο σύστημα, τότε τα μεταδεδομένα αυτά θα πρέπει να ξαναγραφούν από την μνήμη στον δίσκο.



Σχήμα 2: Παράδειγμα ιεραρχίας φακέλων στην μνήμη.

Η κάθε εγγραφή απαιτεί 8 bytes για το ID, 1 byte για τον τύπο, 256 bytes για το όνομα και 8 bytes για τον pointer σε άλλο hashmap. Άρα συνολικά χρειάζονται 273 bytes για κάθε κόμβο. Αν, για παράδειγμα, αποθηκεύσουμε τον κώδικα του Linux Kernel (v6.8.0) στο filesystem μας, ο συνολικός χώρος από τα μεταδεδομένα των nodes θα είναι $273 \times 88982 = 24292086 \approx 23.1667MB$.



Σχήμα 3: Η ιεραρχία κόμβων που περιγράφει το Σχήμα 2.

4 Συναρτήσεις

Οι συναρτήσεις που άλλαξαν είναι οι εξής:

`bb_fullpath`, `bb_getattr`, `bb_mknod`, `bb_mkdir`, `bb_unlink`, `bb_rmdir`, `bb_truncate`, `bb_open`, `bb_read`, `bb_write`, `bb_opendir`, `bb_readdir` `bb_init`.

5 Πειράματα

Για να είμαστε βέβαιοι πως το σύστημα αρχείων λειτουργεί σωστά, χρησιμοποιούμε κάποια αυτοματοποιημένα **functional tests** που ελέγχουν την λειτουργικότητα του συστήματος αρχείων. Έχουμε υλοποιήσει δύο διαφορετικές κατηγορίες tests, η πρώτη ελέγχει τις δομές των blocks και η δεύτερη τις δομές στην ιεραρχία των κόμβων. Για το testing χρησιμοποιείται το Python framework "unittest".

5.1 Έλεγχοι Δομών Blocks

Πείραμα	Σκοπός
Εγγραφή ενός block	Έλεγχος της ορθής εγγραφής και ανάγνωσης ενός block σε ένα αρχείο.
Εγγραφή δέκα blocks	Έλεγχος της ορθής εγγραφής και ανάγνωσης πολλαπλών blocks σε ένα αρχείο.
Εγγραφή δέκα blocks και μισό block	Έλεγχος της ορθής εγγραφής και ανάγνωσης μεγεθών που δεν είναι πολλαπλάσια του μεγέθους του block.
Επανεγγραφή δεδομένων ενός block	Έλεγχος ότι η εγγραφή σε ένα block αντικαθιστά σωστά τα προηγούμενα δεδομένα.
Συμπίεση δεδομένων	Έλεγχος ότι τα δεδομένα συμπιέζονται κατά την εγγραφή και αποσυμπιέζονται ορθά κατά την ανάγνωση.
<code>ftruncate</code> για μείωση αρχείου	Έλεγχος ότι το αρχείο μπορεί να μικρύνει με χρήση <code>ftruncate</code> και να διατηρεί το σωστό περιεχόμενο.
Προσθήκη padding με <code>ftruncate</code>	Έλεγχος ότι όταν το αρχείο μεγαλώνει μέσω <code>ftruncate</code> , τα νέα bytes συμπληρώνονται με μηδενικά.
Defragmentation αποθήκης blocks	Έλεγχος ότι τα blocks ανακατανέμονται σωστά για να μην υπάρχουν κενά μεταξύ τους μετά από διαγραφές.

5.2 Έλεγχοι Δομών Ιεραρχίας Καταλόγων και Αρχείων

Πείραμα	Σκοπός
Δημιουργία και διαγραφή καταλόγου	Έλεγχος της σωστής δημιουργίας και διαγραφής ενός απλού καταλόγου.
Δημιουργία πολλαπλών καταλόγων	Έλεγχος της δημιουργίας πολλών καταλόγων στον ίδιο κατάλογο.
Δημιουργία και διαγραφή καταλόγου και αρχείου	Έλεγχος της δημιουργίας και διαγραφής αρχείων και καταλόγων στον ίδιο κατάλογο.
Δημιουργία πολλών αρχείων σε κατάλογο	Έλεγχος της ορθής διαχείρισης πολλών αρχείων μέσα σε έναν κατάλογο.
Δημιουργία εμφωλευμένων καταλόγων και αρχείου	Έλεγχος της δημιουργίας πολυεπίπεδης ιεραρχίας καταλόγων και προσθήκης αρχείου στο εσωτερικό.
Δημιουργία πολλών καταλόγων και αρχείων	Έλεγχος της δημιουργίας σύνθετης δομής με πολλούς καταλόγους και αρχεία σε διάφορα επίπεδα.

6 Οδηγίες Εκτέλεσης

Οι οδηγίες εκτέλεσης του συστήματος αρχείων και των πειραμάτων βρίσκονται στα Readme αρχεία των φακέλων `filesystem` και `experiments` αντίστοιχα. Αυτές οι οδηγίες συνοψίζονται στις ενότητες 6.1 και 6.2.

6.1 Εκτέλεση Συστήματος Αρχείων

Για την εκτέλεση του `filesystem` σε σύστημα Linux, χρειάζεται αρχικά να είναι εγκατεστημένο το `fuse` και το `libssl`:

```
sudo apt update
sudo apt install pkg-config libssl-dev libfuse-dev
```

Έπειτα, στον φάκελο `filesystem` εκτελούμε την εντολή

```
make
```

Το `filesystem` χρειάζεται δύο φακέλους για να εκτελεστεί, έναν φάκελο που θα αποθηκεύονται τα μεταδεδομένα και έναν που θα χρησιμοποιεί ο χρήστης.

```
mkdir -p rootdir
mkdir -p mountdir
```

Στην συνέχεια εκτελούμε το `executable`, με ορίσματα τον φάκελο των μεταδεδομένων και του χρήστη:

```
./bbfs ./rootdir ./mountdir
```

Αφού χρησιμοποιήσουμε το `filesystem`, τέλος κάνουμε `unmount` με την εντολή:

```
fusermount -u ./mountdir
```

6.2 Εκτέλεση Πειραμάτων

Τα πειράματα εκτελούνται με την παρακάτω εντολή στον φάκελο `experiments`:

```
python3 test.py
```